Symposium AI

Local Real-Time Voice Intelligence & Copilot - Engineering Project Report

Project:	Symposium AI	**Target OS:**	Windows 10 / 11 (64-bit)
Runtime Stack:	Python 3.12, CustomTkinter, SoundDevice	**Inference Engine:**	faster-whisper (CUDA) + Ollama
Repository:	github.com/Asterioxer/symposium-ai	**Execution Mode:**	100% Offline / Local Execution

1. Executive Summary

Symposium AI is an offline, privacy-first desktop application designed to capture speech audio, transcribe questions using a local Speech-to-Text (STT) model, and progressively stream intelligent answers in real time using a locally hosted Large Language Model (LLM).

Unlike cloud-tethered voice assistants, Symposium AI executes all audio processing, speech recognition, and generative AI inference entirely on consumer hardware. This architecture eliminates data exfiltration risks, removes recurring API costs, and delivers sub-second response latencies suitable for live meetings, interviews, and technical discussions.

2. System Architecture

The application is engineered into four decoupled subsystems operating asynchronously across background threads to maintain 60 FPS GUI responsiveness:

Layer	Technology	Role & Key Mechanics
Audio Capture	sounddevice / PortAudio	Captures microphone input at 16 kHz mono float32. Computes live RMS power levels for dynamic UI volume meters.
Speech-to-Text	faster-whisper (CTranslate2)	Performs in-memory transcription directly from float32 NumPy arrays on NVIDIA CUDA (float16) with automatic CPU int8 fallback.
Local LLM	Ollama Client API	Connects to local daemon on localhost:11434. Streams output tokens progressively via generator yields with bounded 2048 context windows.
User Interface	CustomTkinter	Non-blocking desktop interface. Consumes worker thread events via a thread-safe FIFO queue at 30ms intervals.

3. Engineering Challenges & Resolved Edge Cases

3.1 Windows CUDA DLL Discovery (cuBLAS Resolution)

During initial initialization of faster-whisper on Windows, CTranslate2 reported that cublas64_12.dll could not be loaded, even though NVIDIA CUDA drivers were installed. Investigation revealed that the DLL resides in the PyTorch virtual package directory (torch/lib).

Solution: The application dynamically locates the PyTorch library root at startup and calls os.add_dll_directory() while prepending the path to os.environ['PATH']. Additionally, a warm-up dummy encode is executed during initialization; if any CUDA driver fault occurs, the engine falls back to CPU int8 without application disruption.

3.2 Context Window Memory Saturation & LLM Latency

Certain local models (e.g., Qwen 3.5 variants) default to a 256K token context window. In Ollama, allocating a 256K KV cache required 13 GB of VRAM/RAM, causing severe thrashing and stalling first-token latency by over 30 seconds.

Solution: Constrained all Ollama chat sessions to num_ctx: 2048, reducing the active model footprint to ~1.1 GB and enabling immediate sub-second token streaming. Furthermore, lightweight direct-response models (such as dolphinlabs/llama3.2-3b-uncensored) are dynamically sorted to the top of the model selection menu.

3.3 Laptop Microphone Array Sensitivity & VAD Filter Dropout

Integrated digital microphone arrays on Windows often capture audio at low amplitudes (peak amplitude < 0.005). Aggressive Voice Activity Detection (VAD) algorithms treated quiet speech as background noise.

Solution: Implemented dynamic peak normalization scaling in stt_engine.py to bring conversational audio up to ~0.85 peak amplitude before transcription. A two-pass transcription mechanism was also added: if the VAD pass filters out a soft utterance, a non-VAD fallback pass processes the audio directly.

4. Performance Benchmarks & Hardware Resource Consumption

Empirical benchmarks recorded on a Windows 11 host with an Intel processor and NVIDIA GPU:

Metric / Benchmark	Measured Value	Operational Impact
STT Transcription Latency	0.31 seconds (CUDA fp16)	Transcribes 4.0s conversational audio clip in near real-time.
STT Model VRAM Usage	~75 MB (Whisper tiny.en)	Negligible GPU memory overhead; runs alongside larger LLMs.
LLM First-Token Latency	< 0.8 seconds (Llama 3.2 3B)	Progressive word-by-word streaming begins almost immediately.
LLM Token Generation Rate	~35 - 45 tokens / second	Smooth streaming output at faster-than-reading speeds.
GUI Mainloop Latency	< 16 ms (60+ FPS UI)	Zero interface freeze during simultaneous recording and STT.

5. File Manifest & Module Breakdown

Filename	Lines	Description
app.py	~520	Main CustomTkinter desktop UI, thread-safe queue controller, and state manager.
audio_recorder.py	~80	Non-blocking microphone stream using sounddevice with real-time RMS meter.
stt_engine.py	~120	Whisper STT wrapper with DLL discovery, gain scaling, and two-pass VAD.
llm_engine.py	~110	Ollama streaming client with model sorting and reasoning token separation.
test_pipeline.py	~50	Automated non-GUI test script validating audio, STT, and LLM streaming.
run.bat / run.ps1	~12	Windows launchers that automatically start the local Ollama daemon if inactive.

6. Verification & Automated Testing

The project includes an automated end-to-end test suite (test_pipeline.py) that validates all subsystems without requiring interactive manual user input:

- **Audio Input:** Initializes PortAudio stream and checks device descriptors.
- **Synthetic STT Verification:** Feeds a standardized 16 kHz WAV utterance (test_speech.wav) into the engine and asserts exact transcript matching.
- **Ollama Stream Validation:** Sends prompt to local daemon, verifying chunk-by-chunk generator yields and connectivity.

Test Suite Output Log:

```
Testing audio recorder...
Audio recorder ready.

Testing Whisper STT...
STT engine ready: Whisper tiny.en (CUDA float16)
Sample transcription: 'What is artificial intelligence and machine learning?'

Testing local Ollama...
Available models: ['dolphinlabs/llama3.2-3b-uncensored:latest', 'qwen3.5:4b', ...]
Testing stream with dolphinlabs/llama3.2-3b-uncensored:latest...
High-level, versatile, easy-to-learn programming language.

Pipeline test complete.
```

7. Future Roadmap

1. **System Loopback Audio:** Integrate WASAPI loopback capture to transcribe meeting interlocutors from speakers simultaneously with user mic.
2. **Local Knowledge Base (RAG):** Embed a local vector index (e.g., ChromaDB/LanceDB) to ground answers in user meeting briefs.
3. **Multi-Lingual Transcription:** Support dynamic language detection and auto-translation with larger Whisper checkpoints.